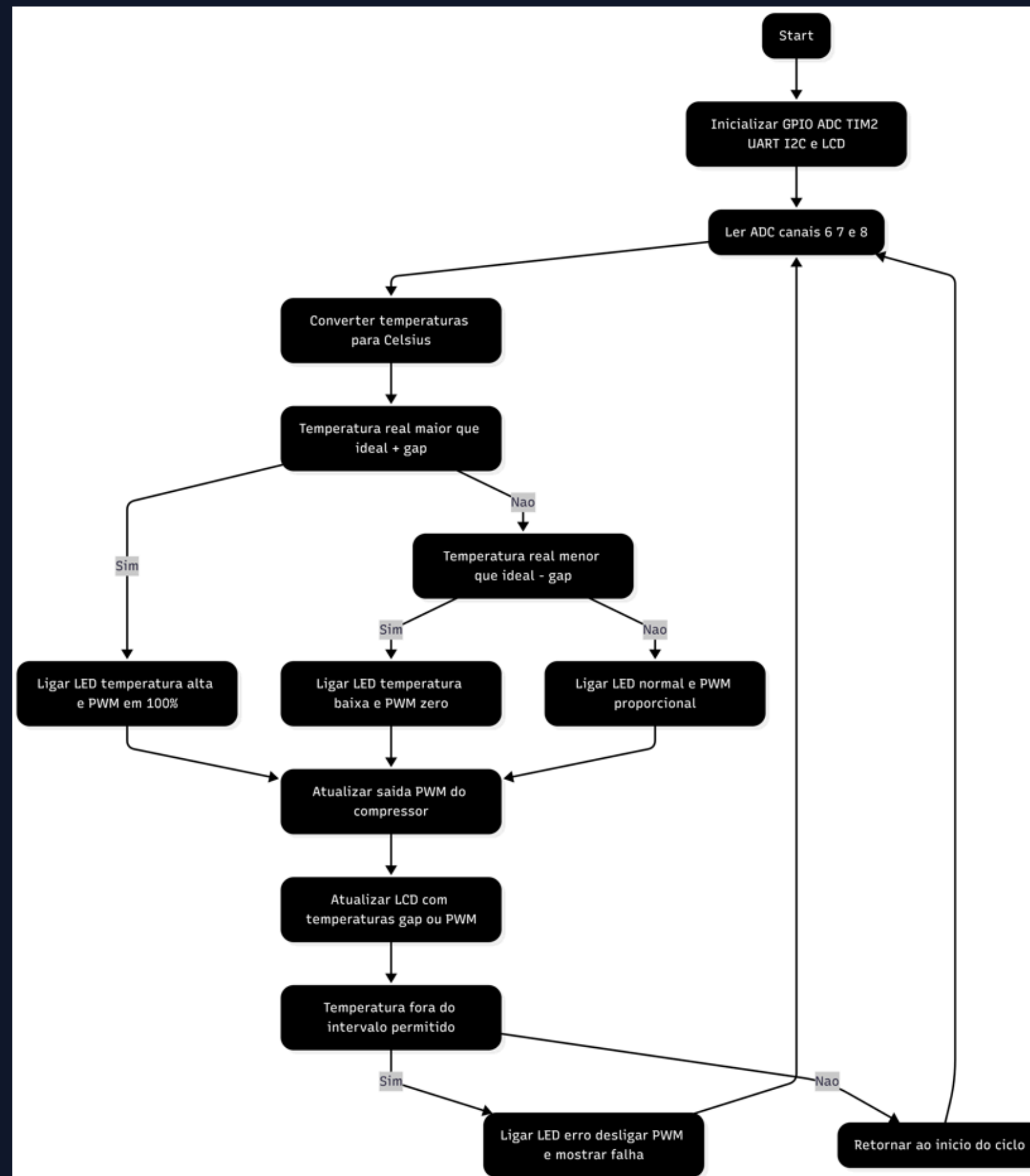


Controlador digital de temperatura

- Gohan Neves
- Pedro Affonso Lopes de Castro
- Rodrigo Lopes Soares Teixeira

Fluxograma de funcionamento



Análise de pinos no STMCubeIDE

- PA6, PA7 E PB0 são canais ADC.
- PA0, PA1, PA2, PA3 e PA4 são LEDS.
- PA11 e PA12 são botões.
- PA5 é um PWM.
- PB6 e PB7 são as conexões com o display.

Watch window associada:

Expression	Type	Value
☒= unidade_temperatura	uint8_t	0 '\0'
☒= temperatura_realc	float	0
☒= temperatura_idealc	float	0
☒= timer1	uint32_t	0
☒= timer2	uint32_t	0
☒= temperatura_idealf	float	0
☒= temperaturaidealk	float	0
☒= gap_temperaturac	float	0



Canais ADC (Potenciômetros)

```
sConfig.Channel = ADC_CHANNEL_8;      // potenciômetro 3 (gap)
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, 100);
adc3 = HAL_ADC_GetValue(&hadc1);
HAL_ADC_Stop(&hadc1);

tensao1 = (float)adc1;
tensao2 = (float)adc2;
tensao3 = (float)adc3;
}
```

```
temperatura_realc    = (tensao1 / 4095.0) * 100.
temperatura_idealc   = (tensao2 / 4095.0) * 100.
gap_temperaturac     = (tensao3 / 4095.0) * 100.
```

● LEDs

```
HAL_GPIO_WritePin(GPIOA, LED_TEMP_BAIXA, 0);
HAL_GPIO_WritePin(GPIOA, LED_COMPRESSOR, 0);
HAL_GPIO_WritePin(GPIOA, LED_NORMAL, 0);
HAL_GPIO_WritePin(GPIOA, LED_ERRO, 0);

if (temperatura_realc > temperatura_idealc + gap_temperaturac){
    HAL_GPIO_WritePin(GPIOA, LED_TEMP_ALTA, 1);
    HAL_GPIO_WritePin(GPIOA, LED_COMPRESSOR, 1);
    ...
}
else if (temperatura_realc < temperatura_idealc - gap_temperaturac){
    HAL_GPIO_WritePin(GPIOA, LED_TEMP_BAIXA, 1);
    ...
}
else {
    HAL_GPIO_WritePin(GPIOA, LED_NORMAL, 1);
    ...
}
```


Botão STOP (PA11) - Segurança

```
HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

if (GPIO_Pin == GPIO_PIN_11)
{
    HAL_GPIO_WritePin(GPIOA,
                      GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
                      GPIO_PIN_3 | GPIO_PIN_4,
                      GPIO_PIN_SET);

    lcd_clear();
    lcd_put_cur(0, 0);
    lcd_send_string("erro");
    lcd_put_cur(1, 0);
    lcd_send_string("erro");
    HAL_Delay(500);
}
```

```
while (1)
{
    // trava o código aqui
}
}
```

Botão (PA12) - Temperatura no menu

```
temperatura_idealf = (temperatura_idealc * 9.0/5.0) + 32.0; // °F
temperaturaidealk  = temperatura_idealc + 273.15;          // K
```

```
if (GPIO_Pin == GPIO_PIN_12)
{
    unidade_temperatura++;
    if (unidade_temperatura > 2)
        unidade_temperatura = 0;
}
}
```

⚙ Estados do Sistema

```
typedef enum {  
    ST_EMESPERA, ST_LENDO, ST_PROCESSANDO, ST_CONTROLE,  
    ST_DISPLAY, ST_PROTECAO, ST_MENU, ST_EXIBIR_CELSIUS,  
    ST_EXIBIR_FAHRENHEIT, ST_EXIBIR_KELVIN, ST_ZERAR_TEMP, ST_ERRO  
} state_t;  
  
state_t estado_atual = ST_EMESPERA;
```


Exibição no display

```
if (unidade_temperatura == 0) {          // Celsius
    temp_real_int  = (int)(temperatura_realc * 100);
    temp_ideal_int = (int)(temperatura_idealc * 100);
    gap_int        = (int)(gap_temperaturac * 100);
    unidade_char[0] = 'C';
}
else if (unidade_temperatura == 1) {     // Fahrenheit
    float temp_real_f  = (temperatura_realc * 9.0/5.0) + 32.0;
    float temp_ideal_f = (temperatura_idealc * 9.0/5.0) + 32.0;
    float gap_f        = (gap_temperaturac * 9.0/5.0);

    temp_real_int  = (int)(temp_real_f * 100);
    temp_ideal_int = (int)(temp_ideal_f * 100);
    gap_int        = (int)(gap_f * 100);
    unidade_char[0] = 'F';
}
else {                                    // Kelvin
    float temp_real_k  = temperatura_realc + 273.15;
    float temp_ideal_k = temperatura_idealc + 273.15;
    float gap_k        = gap_temperaturac;

    temp_real_int  = (int)(temp_real_k * 100);
    temp_ideal_int = (int)(temp_ideal_k * 100);
    gap_int        = (int)(gap_k * 100);
    unidade_char[0] = 'K';
}
```

```
    atual == 0) {
        clear();
        out_cur(0, 0);
        printf(lcd_buffer, "Real:%d.%02d%c",
                temp_real_int/100, temp_real_int%100, unidade_char[0]);
        send_string(lcd_buffer);

        out_cur(1, 0);
        printf(lcd_buffer, "Ideal:%d.%02d%c",
                temp_ideal_int/100, temp_ideal_int%100, unidade_char[0]);
        send_string(lcd_buffer);
    }
```